

LAPORAN PRAKTIKUM

STRUKTUR DATA

Single Linked List

disusun Oleh:

RIFKI MAULANA

NIM 2511533007

DOSEN PENGAMPU : DR. WAHYUDI, S.T, M.T

ASISTEN PRAKTIKUM : ZAHARAN AZARIA ANVAYA



FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

PADANG, 7 MEI 2026

KATA PENGANTAR

Puji syukur kehadiran Allah SWT yang telah memberikan rahmat, taufik, dan hidayahNya sehingga penulis dapat menyelesaikan laporan praktikum mata kuliah Struktur data dengan baik dan tepat waktu. Laporan ini disusun untuk memenuhi salah satu tugas praktikum pada pertemuan kelima dengan judul “**Single Linked List**”. Melalui penyusunan laporan ini, penulis berharap dapat memberikan gambaran mengenai cara pemrograman struktur data Single Linked List.

Penulis menyadari bahwa laporan ini masih jauh dari sempurna. Oleh karena itu, penulis sangat mengharapkan kritik dan saran yang membangun demi perbaikan laporan maupun pemahaman di masa yang akan datang. Akhir kata, penulis mengucapkan terima kasih kepada dosen pengampu serta semua pihak yang telah memberikan bimbingan, arahan, dan bantuan hingga laporan ini dapat terselesaikan. Semoga laporan ini dapat bermanfaat bagi pembaca.

Padang, 7 Mei 2026

Penulis

DAFTAR ISI

LAPORAN PRAKTIKUM.....	1
KATA PENGANTAR.....	i
DAFTAR ISI.....	ii
DAFTAR GAMBAR.....	iii
BAB I	1
1.1 Latar Belakang	1
1.2 Tujuan Praktikum.....	1
1.3 Manfaat Praktikum	1
BAB II PEMBAHASAN	2
2.1 Dasar Teori	2
2.2 Kode Program.....	2
2.2.1 NodeSLL.....	2
2.2.2 TambahSLL.....	2
2.2.3 PencarianSLL.....	4
2.2.4 HapusSLL.....	5
BAB III KESIMPULAN DAN SARAN.....	8
3.1 Kesimpulan	8
DAFTAR PUSTAKA.....	9

DAFTAR GAMBAR

gambar 1.....	4
gambar 2.....	5
gambar 3.....	7
gambar 4 tampilan kode.....	13
gambar 5 output jika memilih pilihan 1.....	14
gambar 6 output jika memilih pilihan 2.....	14
gambar 7 output jika memilih pilihan 3.....	14
gambar 8 output jika memilih pilihan 4.....	14
gambar 9 output jika memilih pilihan 5.....	15
gambar 10 output jika memilih pilihan bukan dari 1-6	15
gambar 11 output jika memilih pilihan 6.....	15

BAB I

PENDAHULUAN

1.1 Latar Belakang

Struktur data adalah cara untuk menyimpan dan mengatur data sehingga Anda dapat menggunakannya secara efisien. Linked list adalah struktur data linear yang terdiri dari kumpulan elemen yang disebut node, di mana setiap node menyimpan dua bagian utama yaitu data dan referensi (pointer) ke node berikutnya. Single linked list adalah jenis linked list paling sederhana, di mana setiap node hanya memiliki satu pointer yang menunjuk ke node berikutnya.

1.2 Tujuan Praktikum

- Memahami konsep *Abstract Data Type (ADT) Single Linked List* dalam pemrograman.
- Mampu merancang dan mengimplementasikan *Single Linked List* menggunakan bahasa Java.

1.3 Manfaat Praktikum

- Menambah pemahaman mengenai penerapan *Single Linked List* dalam kasus kehidupan nyata.
- Melatih kemampuan membuat program yang terstruktur.
- Membantu memahami penggunaan class sebagai representasi objek di Java.

BAB II PEMBAHASAN

2.1 Dasar Teori

Single Linked List adalah struktur data fundamental, terdiri dari node di mana setiap node berisi field data dan referensi ke node berikutnya dalam linked list. Node berikutnya setelah node terakhir adalah null, yang menunjukkan akhir dari list. Linked list mendukung operasi penyisipan dan penghapusan yang efisien. Beberapa contoh operasi dasar di linked list yaitu traversal, insertion, deletion, searching, updating, dan reversal.

2.2 Kode Program

2.2.1 NodeSLL

```
package pekan5_2511533007;

public class NodeSLL_2511533007 {
    // node bagian data
    int data_3007;
    //pointer ke node berikutnya
    NodeSLL_2511533007 next_3007;
    // konstruktor menginisialisasi node dengan data
    public NodeSLL_2511533007 (int data_3007)
    {
        this.data_3007 = data_3007;
        this.next_3007 = null;
    }
}
```

Kode Program 2.1

Program ini berfungsi ADT dari sebuah Node. Setiap Node dianalogikan sebagai sebuah kotak yang memiliki dua bagian: satu tempat untuk menyimpan data angka (data_3007) dan satu lagi sebagai Pointer (next_3007) yang berfungsi menyimpan alamat Node berikutnya. Tanpa kode program ini, kita tidak bisa membentuk rangkaian Linked List.

2.2.2 TambahSLL

```
package pekan5_2511533007;

public class TambahSLL_2511533007 {
    public static NodeSLL_2511533007 insertAtFront (NodeSLL_2511533007 head_3007, int value_3007)
    {
        NodeSLL_2511533007 new_node_3007 = new NodeSLL_2511533007(value_3007);
        new_node_3007.next_3007 = head_3007;
        return new_node_3007;
    }
    // fungsi menambahkan node di akhir SLL
    public static NodeSLL_2511533007 insertAtEnd(NodeSLL_2511533007 head_3007, int value_3007) {
        // buat sebuah node dengan sebuah nilai
        NodeSLL_2511533007 newNode_3007 = new NodeSLL_2511533007(value_3007);
        //jika list kosong maka node jadi head
        if (head_3007 == null) {
            return newNode_3007;
        }
        // simpan head ke variable sementara
        NodeSLL_2511533007 last_3007 = head_3007;
        //telusuri ke node akhir
        while (last_3007.next_3007 != null) {
```

```

        last_3007 = last_3007.next_3007;
    }
    //ubah pointer
    last_3007.next_3007 = newNode_3007;
    return head_3007;
}
static NodeSLL_2511533007 GetNode_3007(int data_3007) {
    return new NodeSLL_2511533007(data_3007);
}
static NodeSLL_2511533007 insertPos_3007(NodeSLL_2511533007 headNode_3007, int position_3007,
int value_3007) {
    NodeSLL_2511533007 head_3007 = headNode_3007;
    if (position_3007 < 1)
        System.out.println("Invalid position");
    if (position_3007 == 1) {
        NodeSLL_2511533007 new_node_3007 = new NodeSLL_2511533007(value_3007);
        new_node_3007.next_3007 = head_3007;
        return new_node_3007;
    } else {
        while (position_3007-- != 0) {
            if (position_3007 == 1) {
                NodeSLL_2511533007 newNode_3007 = GetNode_3007(value_3007);
                newNode_3007.next_3007 = headNode_3007.next_3007;
                headNode_3007.next_3007 = newNode_3007;
                break;
            }
            headNode_3007 = headNode_3007.next_3007;
        }
        if (position_3007 != 1)
            System.out.println("Posisi di luar jangkauan");
    }
    return head_3007;
}
public static void printList_3007(NodeSLL_2511533007 head_3007) {
    NodeSLL_2511533007 curr_3007 = head_3007;
    while (curr_3007.next_3007 != null) {
        System.out.print(curr_3007.data_3007+"-->");
        curr_3007 = curr_3007.next_3007;
    }
    if (curr_3007.next_3007 == null) {
        System.out.print(curr_3007.data_3007);
        System.out.println();
    }
}

public static void main(String[] args) {
    // buat linked list 2->3->5->6
    NodeSLL_2511533007 head_3007 = new NodeSLL_2511533007 (2);
    head_3007.next_3007 = new NodeSLL_2511533007(3);
    head_3007.next_3007.next_3007 = new NodeSLL_2511533007(5);
    head_3007.next_3007.next_3007.next_3007 = new NodeSLL_2511533007(6);
    // cetak list asli
    System.out.print("Senarai berantai awal:");
    printList_3007(head_3007);
    // tambahkan node baru di depan
    System.out.print("tambah 1 simpul di depan: ");
    int data = 1;
    head_3007 = insertAtFront(head_3007, data);
    // cetak update list
    printList_3007(head_3007);
    // tambahkan node baru di belakang
    System.out.print("tambah 1 simpul di belakang: ");
    int data2 = 7;
    head_3007 = insertAtEnd(head_3007, data2);
    // cetak update list
    printList_3007(head_3007);
    System.out.print("tambah 1 simpul ke data 4: ");
    int data3 = 4;
    int pos = 4;
    head_3007 = insertPos_3007(head_3007, pos, data3);
    // cetak update list
    printList_3007(head_3007);
}
}

```

Kode Program 2.2

Kode ini berfungsi untuk menyisipkan Node baru ke dalam rangkaian list yang sudah ada. Ada tiga cara kerja yang diterapkan: meletakkan Node baru di posisi paling depan sebagai Head baru, menaruhnya di akhir list setelah melakukan penelusuran, atau menyisipkannya di posisi tengah sesuai nomor urutan yang diinginkan. Intinya, kode ini mengatur ulang arah Pointer agar anggota baru bisa masuk ke dalam barisan dengan benar.

```
Senarai berantai awal:2-->3-->5-->6
tambah 1 simpul di depan: 1-->2-->3-->5-->6
tambah 1 simpul di belakang: 1-->2-->3-->5-->6-->7
tambah 1 simpul ke data 4: 1-->2-->3-->4-->5-->6-->7
```

gambar 1

2.2.3 PencarianSLL

```
package pekan5_2511533007;

public class PencarianSLL_2511533007 {
    static boolean searchKey_3007(NodeSLL_2511533007 head_3007, int key_3007) {
        NodeSLL_2511533007 curr_3007 = head_3007;
        while (curr_3007 != null) {
            if (curr_3007.data_3007 == key_3007)
                return true;
            curr_3007 = curr_3007.next_3007;
        }
        return false;
    }

    public static void traversal_3007(NodeSLL_2511533007 head_3007) {
        //mulai dari head
        NodeSLL_2511533007 curr_3007 = head_3007;
        //telusuri sampai pointer null
        while (curr_3007 != null) {
            System.out.print(" " + curr_3007.data_3007);
            curr_3007 = curr_3007.next_3007;
        }
        System.out.println();
    }

    public static void main(String[] args) {
        NodeSLL_2511533007 head_3007 = new NodeSLL_2511533007(14);
        head_3007.next_3007 = new NodeSLL_2511533007(21);
        head_3007.next_3007.next_3007 = new NodeSLL_2511533007(13);
        head_3007.next_3007.next_3007.next_3007 = new NodeSLL_2511533007(30);
        head_3007.next_3007.next_3007.next_3007.next_3007 = new NodeSLL_2511533007(10);
        System.out.print("Penelusuran SLL : ");
        traversal_3007(head_3007);
        //data yang akan dicari
        int key_3007 = 30;
        System.out.print("cari data " + key_3007 + " = ");
        if (searchKey_3007(head_3007, key_3007))
            System.out.println("ketemu");
        else
            System.out.println("tidak ada");
    }
}
```

Kode Program 2.3

Bagian ini berfungsi untuk membaca atau memeriksa isi dari Singly Linked List. Terdapat operasi Traversal untuk mengunjungi setiap Node dari awal hingga akhir yang berguna untuk menampilkan isinya. Selain itu, ada fungsi pencarian yang bekerja dengan cara mengecek data pada setiap Node satu per satu (linear search) hingga menemukan nilai yang cocok dengan key yang dicari cari.

```
Penelusuran SLL : 14 21 13 30 10
cari data 30 = ketemu
```

gambar 2

2.2.4 HapusSLL

```
package pekan5_2511533007;

public class HapusSLL_2511533007 {
    //fungsi untuk menghapus head
    public static NodeSLL_2511533007 deleteHead_3007 (NodeSLL_2511533007 head_3007) {
        //jika SLL kosong
        if (head_3007 == null)
            return null;
        //pindahkan head ke node berikutnya
        head_3007 = head_3007.next_3007;
        //return head baru
        return head_3007;
    }

    //fungsi menghapus node terakhir SLL
    public static NodeSLL_2511533007 removeLastNode_3007 (NodeSLL_2511533007 head_3007) {
        //jika list kosong, return null
        if (head_3007 == null) {
            return null;
        }
        //jika list satu node, hapus node dan return null
        if (head_3007.next_3007 == null) {
            return null;
        }
        //temukan node terakhir ke dua
        NodeSLL_2511533007 secondLast_3007 = head_3007;
        while (secondLast_3007.next_3007.next_3007 != null) {
            secondLast_3007 = secondLast_3007.next_3007;
        }
        //hapus node terakhir
        secondLast_3007.next_3007 = null;
        return head_3007;
    }

    //fungsi menghapus node di posisi tertentu
    public static NodeSLL_2511533007 deleteNode_3007 (NodeSLL_2511533007 head_3007, int position_3007)
    {
        NodeSLL_2511533007 temp_3007 = head_3007;
        NodeSLL_2511533007 prev_3007 = null;
        //jika linked list null
        if (temp_3007 == null)
            return head_3007;
        //kasus 1 : head di hapus
        if (position_3007 == 1) {
            head_3007 = temp_3007.next_3007;
            return head_3007;
        }
        //kasus 2: menghapus node di tengah
        //telusuri ke node yang di hapus
        for (int i = 1 ; temp_3007 != null && i < position_3007; i++) {
            prev_3007 = temp_3007;
            temp_3007 = temp_3007.next_3007;
        }
    }
}
```

```

        //jika ditemukan, hapus node
        if (temp_3007 != null) {
            prev_3007.next_3007 = temp_3007.next_3007;
        } else {
            System.out.println("Data tidak ada");
        }
        return head_3007;
    }

    //fungsi mencetak SLL
    public static void printList_3007 (NodeSLL_2511533007 head_3007) {
        NodeSLL_2511533007 curr_3007 = head_3007;
        while (curr_3007.next_3007 != null) {
            System.out.print(curr_3007.data_3007 + "-->");
            curr_3007 = curr_3007.next_3007;
        }
        if (curr_3007.next_3007 == null) {
            System.out.print(curr_3007.data_3007);
        }
        System.out.println();
    }

    //kelas main
    public static void main(String[] args) {
        //buat SLL 1->2->3->4->5->6->null
        NodeSLL_2511533007 head_3007 = new NodeSLL_2511533007 (1);
        head_3007.next_3007 = new NodeSLL_2511533007 (2);
        head_3007.next_3007.next_3007 = new NodeSLL_2511533007 (3);
        head_3007.next_3007.next_3007.next_3007 = new NodeSLL_2511533007 (4);
        head_3007.next_3007.next_3007.next_3007.next_3007 = new NodeSLL_2511533007 (5);
        head_3007.next_3007.next_3007.next_3007.next_3007.next_3007 = new NodeSLL_2511533007 (6);

        //cetak list awal
        System.out.println ("list awal: ");
        printList_3007(head_3007);

        //hapus head
        head_3007 = deleteHead_3007 (head_3007);
        System.out.println ("list setelah head dihapus: ");
        printList_3007(head_3007);

        //hapus node terakhir
        head_3007 = removeLastNode_3007 (head_3007);
        System.out.println ("list setelah simpul terakhir di hapus: ");
        printList_3007(head_3007);

        //Deleting node at position 2
        int position_3007 = 2;
        head_3007 = deleteNode_3007 (head_3007, position_3007);
        //print list after deletion
        System.out.println ("list setelah posisi 2 dihapus: ");
        printList_3007(head_3007);
    }
}

```

Kode Program 2.4

Program terakhir ini digunakan untuk memutus atau mengeluarkan Node tertentu dari rangkaian list. Caranya adalah dengan mengubah arah Pointer pada Node sebelumnya agar langsung melompati Node yang ingin dihapus dan menunjuk ke Node setelahnya. Kita bisa menghapus bagian Head, menghapus Node di posisi paling belakang, ataupun menghapus Node di posisi spesifik sesuai kebutuhan manajemen data kita.

```
list awal:  
1-->2-->3-->4-->5-->6  
list setelah head dihapus:  
2-->3-->4-->5-->6  
list setelah simpul terakhir di hapus:  
2-->3-->4-->5  
list setelah posisi 2 dihapus:  
2-->4-->5
```

gambar 3

BAB III

KESIMPULAN DAN SARAN

3.1 Kesimpulan

Single Linked List merupakan struktur data linear yang tersusun atas rangkaian unit mandiri yang disebut Node, di mana masing-masing Node menyimpan data dan sebuah Pointer menuju Node berikutnya. Berbeda dengan array, SLL bersifat dinamis sehingga memungkinkan manajemen memori yang lebih efisien karena ukuran list dapat bertambah atau berkurang secara fleksibel saat program berjalan. Struktur ini selalu diawali oleh sebuah Head sebagai titik masuk utama dan diakhiri dengan pointer yang menunjuk ke nilai null, yang menandakan batas akhir dari seluruh rangkaian data.

Implementasi SLL mencakup berbagai operasi dasar seperti Insertion (penambahan), Deletion (penghapusan), serta Traversal dan Searching untuk penelusuran data. Keunggulan utama dari struktur ini terletak pada fleksibilitas manipulasi datanya. Proses penambahan atau penghapusan elemen di posisi mana pun dapat dilakukan secara mudah hanya dengan mengatur ulang arah Pointer antar Node tanpa harus menggeser elemen lainnya. Dengan demikian, SLL menjadi Abstract Data Type (ADT) dasar yang sangat penting dalam memahami Struktur data.

DAFTAR PUSTAKA

- [1] Telkom University Jakarta, "Linked List: Single, Double, dan Circular Linked List," [Daring]. Tersedia pada: <https://jakarta.telkomuniversity.ac.id/linked-list-single-double-dan-circular-linked-list/>. [Diakses: 07-Mei-2026].
- [2] GeeksforGeeks, "Singly Linked List Tutorial," [Daring]. Tersedia pada: <https://www.geeksforgeeks.org/dsa/singly-linked-list-tutorial/>. [Diakses: 07-Mei-2026].

Laporan Tugas

A. Kode ADT

```
package pekan5_2511533007;

public class Pasien_2511533007 {

    // Atribut data pasien
    private String namaPasien_3007;
    private String penyakit_3007;
    private int nomorAntrian_3007;
    Pasien_2511533007 next_3007;

    // Constructor menginisialisasi semua atribut
    public Pasien_2511533007(String namaPasien_3007, String penyakit_3007, int nomorAntrian_3007) {

        this.namaPasien_3007 = namaPasien_3007;
        this.penakit_3007 = penyakit_3007;
        this.nomorAntrian_3007 = nomorAntrian_3007;
        this.next_3007 = null;

    }

    // Getter untuk setiap atribut
    public String getNamaPasien_3007() {
        return namaPasien_3007;
    }
    public String getPenyakit_3007() {
        return penyakit_3007;
    }
    public int getNomorAntrian_3007() {
        return nomorAntrian_3007;
    }
    public Pasien_2511533007 getNext_3007() {
        return next_3007;
    }

    // Setter untuk setiap atribut
    public void setNamaPasien_3007(String namaPasien_3007) {
        this.namaPasien_3007 = namaPasien_3007;
    }
    public void setPenyakit_3007(String penyakit_3007) {
        this.penakit_3007 = penyakit_3007;
    }
    public void setNomorAntrian_3007(int nomorAntrian_3007) {
        this.nomorAntrian_3007 = nomorAntrian_3007;
    }
    public void setNext_3007(Pasien_2511533007 next_3007) {
        this.next_3007 = next_3007;
    }

}
```

B. Kode Program

```
package pekan5_2511533007;

import java.util.Scanner;

public class RumahSakit_2511533007 {

    // head linked list dan counter nomor antrian
    static Pasien_2511533007 head_3007 = null;
    static int counterAntrian_3007 = 0;

    // fungsi mendaftarkan pasien baru di akhir list (insert at tail)
    public static void daftarkanPasien_3007(String nama_3007, String keluhan_3007) {
        counterAntrian_3007++;
        Pasien_2511533007 newPasien_3007 = new Pasien_2511533007(nama_3007, keluhan_3007,
        counterAntrian_3007);

        // jika list kosong, langsung jadi head
    }
```

```

        if (head_3007 == null) {
            head_3007 = newPasien_3007;
        } else {
            // telusuri ke node paling akhir lalu sambungkan
            Pasien_2511533007 last_3007 = head_3007;
            while (last_3007.next_3007 != null) {
                last_3007 = last_3007.next_3007;
            }
            last_3007.next_3007 = newPasien_3007;
        }
        System.out.println("\nPasien berhasil didaftarkan");
        System.out.println("Nomor Antrian: " + counterAntrian_3007);
    }

    // fungsi memanggil pasien terdepan (delete head)
    public static void panggilPasien_3007() {
        if (head_3007 == null) {
            System.out.println("\nAntrian kosong, tidak ada pasien yang bisa dipanggil.");
            return;
        }
        // tampilkan data lalu geser head ke node berikutnya
        System.out.println("\nMemanggil pasien: ");
        cetakDataPasien_3007(head_3007);
        head_3007 = head_3007.next_3007;
    }

    // fungsi menampilkan seluruh antrian (display)
    public static void tampilkanAntrian_3007() {
        if (head_3007 == null) {
            System.out.println("\nAntrian kosong.");
            return;
        }
        System.out.println("\nDAFTAR ANTRIAN PASIEN");
        Pasien_2511533007 curr_3007 = head_3007;
        int posisi_3007 = 1;

        // telusuri dari head hingga null
        while (curr_3007 != null) {
            System.out.println("Posisi ke " + posisi_3007);
            cetakDataPasien_3007(curr_3007);
            curr_3007 = curr_3007.next_3007;
            posisi_3007++;
        }
        System.out.println("");
    }

    // fungsi mencari pasien berdasarkan nama
    public static void cariPasien_3007(String namaCari_3007) {
        Pasien_2511533007 curr_3007 = head_3007;
        boolean ditemukan_3007 = false;

        // telusuri seluruh list
        while (curr_3007 != null) {
            if (curr_3007.getNamaPasien_3007().equalsIgnoreCase(namaCari_3007)) {
                if (!ditemukan_3007)
                    System.out.println("\nHasil pencarian:");
                cetakDataPasien_3007(curr_3007);
                ditemukan_3007 = true;
            }
            curr_3007 = curr_3007.next_3007;
        }

        if (!ditemukan_3007)
            System.out.println("\nPasien dengan nama \"" + namaCari_3007 + "\" tidak ditemukan.");
    }

    // fungsi mengecek status antrian
    public static void cekStatusAntrian_3007() {
        if (head_3007 == null) {
            System.out.println("\nAntrian kosong, tidak ada pasien saat ini.");
            return;
        }
        // hitung total pasien dengan menelusuri list
    }

```

```

        int total_3007 = 0;
        Pasien_2511533007 curr_3007 = head_3007;
        while (curr_3007 != null) {
            total_3007++;
            curr_3007 = curr_3007.next_3007;
        }
        System.out.println("\nSTATUS ANTRIAN ");
        System.out.println("Total pasien dalam antrian:" + total_3007);
        System.out.println("Pasien terdepan: ");
        cetakDataPasien_3007(head_3007);
        System.out.println("");
    }

    // helper mencetak data satu node pasien
    private static void cetakDataPasien_3007(Pasien_2511533007 p_3007) {
        System.out.println("No. Antrian: " + p_3007.getNomorAntrian_3007());
        System.out.println("Nama: " + p_3007.getNamaPasien_3007());
        System.out.println("Penyakit: " + p_3007.getPenyakit_3007());
        System.out.println("");
    }

    public static void main(String[] args) {
        Scanner scanner_3007 = new Scanner(System.in);
        int pilihan_3007;

        do {
            System.out.println("SISTEM ANTRIAN RUMAH SAKIT");
            System.out.println("");
            System.out.println("1. Daftarkan Pasien (Insert)");
            System.out.println("2. Panggil Pasien (Delete Head)");
            System.out.println("3. Tampilkan Antrian (Display)");
            System.out.println("4. Cari Pasien (Search)");
            System.out.println("5. Cek Status Antrian ");
            System.out.println("6. Keluar ");
            System.out.println("");
            System.out.print("Pilih menu (1-6): ");
            pilihan_3007 = scanner_3007.nextInt();
            scanner_3007.nextLine();

            switch (pilihan_3007) {
                case 1:
                    System.out.print("Masukkan Nama Pasien : ");
                    String nama_3007 = scanner_3007.nextLine();
                    System.out.print("Masukkan Keluhan : ");
                    String keluhan_3007 = scanner_3007.nextLine();
                    daftarkanPasien_3007(nama_3007, keluhan_3007);
                    break;

                case 2:
                    panggilPasien_3007();
                    break;

                case 3:
                    tampilkanAntrian_3007();
                    break;

                case 4:
                    System.out.print("Nama yang dicari: ");
                    String namaCari_3007 = scanner_3007.nextLine();
                    cariPasien_3007(namaCari_3007);
                    break;

                case 5:
                    cekStatusAntrian_3007();
                    break;

                case 6:
                    System.out.println("\nSistem antrian ditutup. Terima kasih.");
                    break;

                default:
                    System.out.println("\nPilihan tidak valid, masukkan angka 1-6.");
            }
        } while (pilihan_3007 != 6);
    }

```



```

    scanner_3007.close();
}
}

```

C. Penjelasan Kode

Kelas pertama Pasien adalah ADT atau Node yang berfungsi menyimpan informasi spesifik setiap pasien seperti nama, keluhan penyakit, dan nomor antrian, serta memiliki atribut next_3007 sebagai pointer ke pasien berikutnya dalam barisan.

Kelas kedua RumahSakit berisi logika dan implementasi ADT.

Pendaftaran (daftarKanPasien): Menerapkan fungsi Insert at Tail. Program akan membuat node baru, lalu menelusuri rantai hingga menemukan ujung (null) untuk menyambungkan pasien baru tersebut.

Pemanggilan (panggilPasien): Menerapkan fungsi Delete Head. Pasien yang berada di posisi paling depan (head) akan ditampilkan datanya, kemudian posisi head digeser ke pasien berikutnya. Sehingga menghapus pasien pertama dari daftar secara otomatis.

Penelusuran (tampilkanAntrian & cekStatus): Melakukan traversal atau pengulangan dari head sampai ujung list untuk menghitung jumlah pasien atau menampilkan urutan antrian secara visual.

Pencarian (cariPasien): Menggunakan metode Linear Search dengan membandingkan nama yang diinput user dengan atribut nama di setiap node selama proses penelusuran.

Interaksi Pengguna: Metode main menyediakan menu untuk di input menggunakan Scanner dan switch case agar pengguna dapat mengoperasikan sistem dengan mudah.

D. Output Kode

```

SISTEM ANTRIAN RUMAH SAKIT

1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar

Pilih menu (1-6):

```

gambar 4 tampilan kode

```
Pilih menu (1-6): 1
Masukkan Nama Pasien : qwer
Masukkan Keluhan      : asd

Pasien berhasil didaftarkan
Nomor Antrian: 1
```

gambar 5 output jika memilih pilihan 1

```
Pilih menu (1-6): 2
|
Memanggil pasien:
No. Antrian: 1
Nama: qwer
Penyakit: asd
```

gambar 6 output jika memilih pilihan 2

```
Pilih menu (1-6): 3

DAFTAR ANTRIAN PASIEN
Posisi ke 1
No. Antrian: 2
Nama: qwertyu
Penyakit: sdfghjk

Posisi ke 2
No. Antrian: 3
Nama: xcvbnm
Penyakit: oiuyt
```

gambar 7 output jika memilih pilihan 3

```
Pilih menu (1-6): 4
Nama yang dicari: qwertyu
|
Hasil pencarian:
No. Antrian: 2
Nama: qwertyu
Penyakit: sdfghjk
```

gambar 8 output jika memilih pilihan 4

```
Pilih menu (1-6): 5
|
STATUS ANTRIAN
Total pasien dalam antrian:2
Pasien terdepan:
No. Antrian: 2
Nama: qwertyu
Penyakit: sdfghjk
```

gambar 9 output jika memilih pilihan 5

```
Pilih menu (1-6): 7

Pilihan tidak valid, masukkan angka 1-6.
SISTEM ANTRIAN RUMAH SAKIT
```

gambar 10 output jika memilih pilihan bukan dari 1-6

```
Pilih menu (1-6): 6
|
Sistem antrian ditutup. Terima kasih.
```

gambar 11 output jika memilih pilihan 6